UCS-503 – Software Engineering

NagrikConnect

A Web-Based Smart City Complaint Management System

Submitted by:

1024170349 [Aarav Bansal]
1024170381 [Pushkar Sheokand]
1024170383 [Tarun Chaudhary]

Group: 3Q34 BE Third Year, CSE

Submitted to:

Paramveer Sidhu

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **5**
- 1.1 Background and Context . . . 5
  - 1.1.1 Problem Statement . . . 5
- 1.2 Project Objectives . . . 5
- 1.3 Project Scope . . . 6

2 Methodology and Design **7**
- 2.1 System Architecture . . . 7
  - 2.1.1 High-Level Design . . . 7
  - 2.1.2 Technical Stack . . . 7
- 2.2 Implementation Details . . . 8
  - 2.2.1 Citizen Access and Complaint Submission . . . 8
  - 2.2.2 Complaint Classification and Routing . . . 8
  - 2.2.3 Location, Attachments, and Timestamps . . . 8
  - 2.2.4 Complaint Status Tracking . . . 8
- 2.3 Data and Privacy Considerations . . . 9

3 Results and Evaluation **11**
- 3.1 Prototype Status . . . 11
- 3.2 Testing Methodology . . . 11
- 3.3 Performance Metrics . . . 12
- 3.4 Analysis and Limitations . . . 12

4 Conclusion **13**
- 4.1 Summary of Work . . . 13
- 4.2 Key Findings . . . 13
- 4.3 Future Work and Recommendations . . . 13
- 4.4 Final Remarks . . . 14

Chapter 1

Introduction

1.1 Background and Context

Urban residents regularly encounter civic issues such as damaged roads, overflowing waste bins, broken streetlights, water leakage, and sanitation problems. Reporting these issues can involve identifying the appropriate authority, providing enough information for the complaint to be understood, and finding out whether action has been taken. When reporting and follow-up processes are fragmented, complaints may be difficult to track and citizens may have limited visibility into their progress.

NagrikConnect is proposed as a web-based civic complaint management system intended to make the reporting and tracking process more structured. It provides a common interface through which citizens can submit civic complaints and review their status. The system is designed around a clear complaint record containing the issue description, location, submission time, supporting media when available, and status updates. Automated assistance is also planned to help categorize complaints and suggest the responsible department and priority.

1.1.1 Problem Statement

Citizens may not know which department should receive a particular civic complaint, and manual reporting can omit useful details such as the precise location or supporting evidence. In addition, the absence of a simple status-tracking interface can make follow-up inconvenient. The project addresses the need for a unified web interface for lodging, organizing, and tracking civic complaints, with automated classification and routing support as intended capabilities.

1.2 Project Objectives

The project objectives for the prototype are:

1. Design and develop a citizen-facing web interface that supports account access and complaint submission.

2. Capture complaint details, submission date and time, location information where permission is granted, and optional image, audio, or video evidence.

3. Provide a structured complaint record and a citizen-facing view of complaint status.

4. Explore automated prediction of complaint category, responsible department, and priority from the submitted description.

5. Establish a modular foundation that can later support administrative workflows, notifications, analytics, and deployment.

1.3 Project Scope

The MST scope is a prototype of the core citizen complaint workflow and its supporting architecture. Automated classification, location capture, media upload, and status tracking should be described as implemented only to the extent that they have been built and tested. Integration with municipal systems, production-grade security, large-scale deployment, and validation on a representative real-world complaint dataset are outside the scope of an initial prototype.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

NagrikConnect is planned as a web application with a frontend, backend API, data store, and optional machine-learning component. The frontend presents forms and dashboards to citizens. The backend validates requests, applies business rules, manages complaint records, and exposes API endpoints. The database stores user and complaint information. A classification component may analyze complaint text and return a predicted category, department, and priority; these predictions should be treated as suggestions and should be reviewable rather than assumed to be correct in every case.

The intended complaint flow is:

1. A citizen accesses the application and signs in or uses the configured demonstration account.
2. The citizen provides the media followed by location and then describes the complaint with type and priority.
3. The application records the submission time and validates the provided fields.
4. If the classification component is available, the complaint text is passed to it for a suggested category, department, and priority.
5. The complaint is saved and displayed to the citizen with its current status.
6. Authorized staff can view the registered complaint, assign a crew member, view progress and provide comments.

2.1.2 Technical Stack

The intended stack for the web version is:

- **Frontend:** React.js with HTML, CSS, and JavaScript.
- **Backend:** Node.js and Express.js for REST API endpoints and server-side validation.
- **Database:** PostgreSQL for the implementation.

- **Development Tools:** Git for version control, VS Code for coding, ChatGPT and Claude for debugging and error resolution, and a web browser for testing the application.

2.2 Implementation Details

2.2.1 Citizen Access and Complaint Submission

The citizen-facing workflow is intended to provide access to the application and a form for lodging a complaint. The form should collect a concise description of the civic issue and include attachments with precise location. Input validation should check required fields, accepted file types, and file-size limits.

2.2.2 Complaint Classification and Routing

The system categorizes complaints using predefined categories such as roads, sanitation, street lighting, and water supply. Based on the selected category, the complaint is automatically routed to the corresponding department using a predefined mapping. Complaint priority is determined using predefined rules based on the type and severity of the issue. The system also allows authorized personnel to review and modify complaint details when necessary.

2.2.3 Location, Attachments, and Timestamps

Where a citizen grants browser location permission, the application can use the Geolocation API to obtain coordinates. Location access should be optional, clearly explained, and accompanied by a manual-entry fallback. The backend should record the submission timestamp consistently. Media upload should validate file type and size and should avoid exposing uploaded files publicly without authorization.

2.2.4 Complaint Status Tracking

Each complaint have a unique identifier and a status field. A simple lifecycle includes *Complaint Registered*, *Department Assigned*, *In Progress*, and *Resolution and Verification*. The citizen dashboard shows only complaints associated with the authenticated citizen, while any administrative status-change operation require appropriate authorization.

2.3 Data and Privacy Considerations

Complaint descriptions, location coordinates, account details, and attachments may contain personal or sensitive information. The application collects only information needed for handling the complaint, request permission before accessing location, restrict access to complaint records, validate uploads, and avoid placing credentials or secrets in source control. A deployed version would also require secure transport, robust authentication, backup and retention policies, and a review of applicable privacy requirements.

Chapter 3

Results and Evaluation

3.1 Prototype Status

Feature	Status to confirm before submission
Citizen login / demo access	[Implemented]
Complaint submission form	[Implemented]
Department and priority	[Implemented]
Location capture	[Implemented]
Image/audio/video upload	[Implemented]
Complaint status dashboard	[Implemented]
Persistent database storage	[In progress]

Table 3.1: Prototype implementation status checklist

3.2 Testing Methodology

Testing should be carried out at multiple levels:

- **Unit testing:** Validate individual form checks, API handlers, classification logic, and status-transition rules.
- **Integration testing:** Verify that the frontend can submit a complaint to the backend and that valid records are stored and retrieved correctly.
- **Input and error testing:** Test missing fields, invalid file types, oversized uploads, denied location permission, and unavailable classification services.
- **Access-control testing:** Confirm that a citizen cannot view or modify another citizen's complaints and that restricted operations require authorization.
- **User testing:** Ask a small number of test users to submit a sample complaint and report any confusing steps.

3.3 Performance Metrics

No measured performance results are assumed in this draft. Record actual observations from the running prototype. If a metric has not been measured, mark it as *Not measured* rather than inserting an estimated value.

Metric	**Target / method**	**Observed result**
Complaint submission	Verify successful submission of valid complaints	Successfully saved
API response time	Measure response time for repeated requests	Not measured
Department routing	Verify category-to-department mapping	Tested as per predefined rules
Invalid-input handling	Test missing and malformed inputs	Shows error for missing inputs
Status retrieval	Verify saved complaint status is displayed correctly	Successful

Table 3.2: Evaluation measures for the prototype

3.4 Analysis and Limitations

The main value of the proposed system is the consolidation of complaint submission and status tracking in one interface. Automated categorization may reduce the effort required to route complaints, but its reliability cannot be claimed without an appropriate labelled dataset and evaluation. Other limitations may include incomplete department mappings, inaccurate or unavailable location data, file-storage constraints, and the absence of integration with real municipal workflows. These are areas to validate and prioritize during continued development.

Chapter 4

Conclusion

4.1 Summary of Work

NagrikConnect addresses the civic complaint-reporting workflow through a planned web application that combines citizen access, structured complaint submission, location and evidence capture, complaint tracking, and automated routing assistance. The architecture separates the frontend, backend API, persistent storage, and classification logic so that each component can be developed and tested independently. The final submission should distinguish the functionality demonstrated in the current prototype from features that remain planned.

4.2 Key Findings

The design process highlights three important considerations:

- **Structured reporting:** A consistent complaint record can make issue descriptions and follow-up easier to manage.
- **Assisted routing:** Category and department predictions can support triage, but require validation and human oversight.
- **Trust and accountability:** Clear status updates, privacy-conscious location capture, and controlled access are important for a civic service.

4.3 Future Work and Recommendations

1. Complete and test the end-to-end citizen complaint workflow.
2. Connect the application to a persistent database and implement secure authentication and authorization.
3. Build or evaluate a complaint text-classification model using a labelled dataset representative of the intended deployment context.
4. Add notifications, analytics, and duplicate-complaint detection where useful.
5. Conduct usability, security, and performance testing before any real-world deployment.

4.4 Final Remarks

NagrikConnect is intended to make civic issue reporting more accessible and traceable. Its effectiveness will depend not only on the interface, but also on reliable workflows, appropriate data handling, validated classification, and coordination with the departments responsible for resolving reported issues. Further implementation and testing are necessary before making claims about real-world performance.